

CoppeliaSim Tutorial 12

椅子からの立ち上がり動作 (Sit-to-Stand)

Tutorial 11では「足部固定 (CKC)」モデルを作成し、転倒しない環境でスクワットを行いました。今回はその応用として、「椅子」を作成し、そこから立ち上がる **Sit-to-Stand (起立動作)** をシミュレーションします。

立ち上がり動作は、単純な膝の曲げ伸ばしだけでなく、「**重心の前方移動 (離殿)**」というフェーズが重要になります。

目的

1. シーン内に「椅子」オブジェクトを作成する
 2. スクリプトを改良し、「前傾 (重心移動)」→「伸展 (立ち上がり)」の2段階動作を実装する
 3. 足部固定モデルならではの安定した挙動を確認する
-

1. 準備：モデルの読み込み

Tutorial 11 で作成したシーン (`squad.ttt` または保存したファイル) を開いてください。これをベースに改造します。

(もし Tutorial 11 が手元にない場合は、Tutorial 11 の手順に従って RFoot 固定モデルを作成してから戻ってきてください)

2. 椅子の作成

ビジュアル的な目安として、お尻の下に椅子を置きます。※今回のモデルは足が固定されているため、物理的に椅子がお尻を支えるわけではありませんが、動作の開始位置として重要です。

1. **Add > Primitive shape > Cuboid**
 2. サイズ: **X=0.40, Y=0.40, Z=0.45**
 - Tutorial 11 のモデル (Pelvis位置 Z=0.90, 大腿長0.35, 下腿長0.40) から計算すると、膝下の高さは約 0.45m~0.50m程度です。これに合わせて高さを調整します。
 3. 位置: **X=-0.30, Y=0, Z=0.225**
 - 足 (X=0) より後ろ (マイナス方向) に配置します。
 - 高さは椅子の中心位置なので、長さの半分 (0.225) を指定して底面を床に合わせます。
 4. 名前: **Chair**
 5. **Dynamic: OFF (Static)**
 - 動かない障害物として設置します。
 6. **Respondable: OFF**
 - **重要:** 今回はスクリプトで「仮想的な反力」を作るため、物理的な衝突判定はOFFにします。
 - ONのままだと、骨盤と椅子が接触・反発して不要な振動やモータートルクの急上昇 (スパイク) が発生し、正確なデータが取れません。
-

2.5 干渉対策（重要）

股関節を深く曲げると、「**骨盤 (Pelvis)**」と「**大腿 (Thigh)**」が物理的に衝突し、動作がブロックされてしまう場合があります。これを防ぐため、身体パーツ同士の衝突判定を無効化します。

1. Scene Hierarchy で **Pelvis**, **RThigh**, **LThigh** をすべて選択します (Shift/Ctrlキーを使用)。
2. **Scene Object Properties** ダイアログを開きます。
3. **Common** (またはダイアログ上部) にある **Respondable** のチェックを **外して OFF** にします。
 - これにより、パーツ同士が重なっても物理的な反発力が生じず、スムーズに屈曲できるようになります。
 - (足部 **RFoot** など床と接地するパーツは Respondable ON のままにしてください)

3. スクリプトの改良 (Sit-to-Stand)

立ち上がり動作をよりリアルにするため、**RFoot** のスクリプトを更新します。単純に「座る→立つ」ではなく、以下のフェーズに分けます。

1. **Rest (安静位)** : 座っている状態
2. **Flexion (屈曲・前傾)** : 体幹を前に倒し、重心を足の上に乗せる (離殿準備)
3. **Extension (伸展)** : 膝と股関節を伸ばして立ち上がる

スクリプトの書き換え

RFoot の子スクリプトを開き、中身を以下のように書き換えてください。

```
sim = require('sim')
```

```
function sysCall_init() rAnkle = sim.getObject("/:RAnkleJoint") rKnee = sim.getObject("/:RKneeJoint") rHip = sim.getObject("/:RHipJoint") pelvis = sim.getObject("/:Pelvis") -- 力を加えるために取得
```

```
-- グラフ用 (Tutorial 11と同様)
graphHandle = sim.getObject("/:TorqueGraph")
if graphHandle ~= -1 then
    torqueStream = sim.addGraphStream(graphHandle, "Knee Torque", "N*m",
    0, {1, 0, 0})
end

timer = 0
```

```
end
```

```
function sysCall_actuation() dt = sim.getSimulationTimeStep() timer = timer + dt
```

```
-- --- 動作タイミングの設定 (秒) ---
local t0 = 1.0 -- 動作開始 (Lean開始)
local t1 = 1.5 -- ★離殿 (Seat-off) & 立ち上がり開始
-- ユーザー観測値に合わせて 1.5秒 に設定
```

```

-- この時点で supportForce を 0 にし、Phase 2 (Extension) へ移行します
local t2 = 3.5 -- 立ち上がり完了 (直立)

-- --- 仮想的な椅子の反力 (Virtual Chair Reaction Force) ---
local pelvisMass = 20.0 -- 椅子が支える骨盤の質量
local g = 9.81
local supportForce = 0

local targetAnkle = 0 -- 足首 (背屈+/底屈-)
local targetKnee = 0 -- 膝 (伸展0/屈曲-)
local targetHip = 0 -- 股 (屈曲+/伸展0)

-- --- 動作フェーズの設定 ---

-- 初期位置 (座位) :
local sitAnkle = 0 * math.pi / 180
local sitKnee = -90 * math.pi / 180
local sitHip = 90 * math.pi / 180

-- 前傾姿勢 (離殿直前) :
local leanAnkle = 30 * math.pi / 180
local leanKnee = -90 * math.pi / 180
local leanHip = 130 * math.pi / 180

-- 直立姿勢: 全部0
local standAnkle = 0
local standKnee = 0
local standHip = 0

-- --- タイミング制御 ---

if timer < t0 then
    -- 【Phase 0】 安定化: 椅子に座って静止
    targetAnkle = sitAnkle
    targetKnee = sitKnee
    targetHip = sitHip
    supportForce = pelvisMass * g

elseif timer < t1 then
    -- 【Phase 1】 前傾動作 (Lean)
    -- t0(1.0s) ~ t1(1.5s) の間で前傾
    local duration = t1 - t0
    local ratio = (timer - t0) / duration
    targetAnkle = (1 - ratio) * sitAnkle + ratio * leanAnkle
    targetKnee = (1 - ratio) * sitKnee + ratio * leanKnee
    targetHip = (1 - ratio) * sitHip + ratio * leanHip

    -- 前傾中もまだ座っているので、支持力は維持
    supportForce = pelvisMass * g

```

```

elseif timer < t2 then
    -- *** ここが離殿 (Seat-off) ! ***
    -- timer >= t1 (1.5s) になると supportForce は 0 (デフォルト値) になります

    -- 【Phase 2】 立ち上がり (Extension)
    -- t1(1.5s) ~ t2(3.5s) の間で伸展
    local duration = t2 - t1
    local ratio = (timer - t1) / duration
    targetAnkle = (1 - ratio) * leanAnkle + ratio * standAnkle
    targetKnee  = (1 - ratio) * leanKnee  + ratio * standKnee
    targetHip   = (1 - ratio) * leanHip   + ratio * standHip

    -- supportForce = 0 (計算しない)

else
    -- 【Phase 3】 直立保持
    targetAnkle = standAnkle
    targetKnee  = standKnee
    targetHip   = standHip
end

sim.setJointTargetPosition(rAnkle, targetAnkle)
sim.setJointTargetPosition(rKnee, targetKnee)
sim.setJointTargetPosition(rHip, targetHip)

-- 椅子反力
if supportForce > 0 then
    sim.addForceAndTorque(pelvis, {0, 0, supportForce})
end

```

```

end function sysCall_sensing() if graphHandle ~= -1 then local torque = sim.getJointForce(rKnee)
sim.setGraphStreamValue(graphHandle, torqueStream, torque) end end

```

4. 実行と確認

再生ボタンを押してシミュレーションを開始します。

動作のチェックポイント

1. **0~1.0秒**： 椅子に座って静止しているか？
2. **1.0~1.5秒**： **「お辞儀」** をして、重心を前方に移動させているか？
* この期間は短くなりましたが、スムーズに重心移動ができているか確認します。
3. **1.5秒~**： そのままスムーズに立ち上がり、直立するか？
* ここが **「離殿 (Seat-off)」** のタイミングです。

トルクグラフの観察

Tutorial 11同様、膝関節のトルク (Knee Torque) を観察してください。

- * **離殿（1.5秒付近）** でトルクはどう変化しましたか？
- * お尻の支え（supportForce）が消えた瞬間、グラフに急激な変化が見られるはずです。

5. （発展）椅子の高さやタイミングを変えてみる

- * スクリプト内の `t1`（離殿タイミング）を調整して、最適なタイミングを探ってみてください。
- * **椅子の高さ（Z座標）** を低くすると、立ち上がりはどうなりますか？
 - * より深い前傾が必要になるかもしれません。
- * **動作スピード（秒数）** を速くすると、トルクは増えますか？

これで、**「環境（椅子）」** と **「動作（立ち上がり）」** を組み合わせた、より実践的なシミュレーションが完成しました。